

Bangladesh University of Business and Technology



LAB REPORT

Experiment No : 02
Course Title : Algorithms Lab
Course Code : CSE 232
Experiment Name: Lab Assignment 02

Submitted By:

Name : Kawsar Hoque
ID No : 20255103270
Intake : 55
Section : 7
Program : B.Sc. Engg. in CSE

Submitted To:

Name : Sumona Yeasmin

(Lecturer)
Dept. of CSE
**Bangladesh University of Business
& Technology**

Date of Submission : 02/06/2026

1. Bubble Sort

● BubbleSort_270.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8      int arr[n];
9      for (int i = 0; i < n; i++)
10         cin >> arr[i];
11
12     int comparisons = 0;
13     for (int i = 0; i < n-1; i++)
14     {
15         for (int j = 0; j < n-i-1; j++)
16         {
17             comparisons++;
18             if (arr[j] > arr[j+1])
19             {
20                 int temp = arr[j];
21                 arr[j] = arr[j+1];
22                 arr[j+1] = temp;
23             }
24         }
25     }
26     cout << "Sorted: ";
27     for (int i = 0; i < n; i++)
28         cout << arr[i] << " ";
29     cout << endl;
30     cout << "Comparisons: " << comparisons;
31     return 0;
32 }
```

Best Case – Already Sorted

```
C:\WINDOWS\system32\cmd.exe
5
1 2 3 4 5
Sorted: 1 2 3 4 5
Comparisons: 10
c:\Users\Kawsar Hoque\Desktop\CODING KH\BUBT_LAB_ALGO
```

Worst Case – Reverse Sorted

```
C:\WINDOWS\system32\cmd.exe
5
5 4 3 2 1
Sorted: 1 2 3 4 5
Comparisons: 10
c:\Users\Kawsar Hoque\Desktop\CODING KH\BUBT_LAB_ALGO
```

Complexity Analysis

Best Case: $O(n^2)$ – Even when the array is already sorted, Bubble Sort still performs $n(n-1)/2$ comparisons because there is no early-exit condition in this version.

Worst Case: $O(n^2)$ – When the array is in reverse order, every comparison results in a swap, producing the maximum number of operations.

2. Selection Sort

● SelectionSort_270.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8      int arr[n];
9      for (int i = 0; i < n; i++)
10         cin >> arr[i];
11
12     int comparisons = 0;
13     for (int i = 0; i < n-1; i++)
14     {
15         int minIdx = i;
16         for (int j = i+1; j < n; j++)
17         {
18             comparisons++;
19             if (arr[j] < arr[minIdx])
20                 minIdx = j;
21         }
22         int temp = arr[minIdx];
23         arr[minIdx] = arr[i];
24         arr[i] = temp;
25     }
26     cout << "Sorted: ";
27     for (int i = 0; i < n; i++)
28         cout << arr[i] << " ";
29     cout << endl;
30     cout << "Comparisons: " << comparisons;
31     return 0;
32 }
```

Best Case – Already Sorted

```
C:\WINDOWS\system32\cmd.exe
5
1 2 3 4 5
Sorted: 1 2 3 4 5
Comparisons: 10
c:\Users\Kawsar Hoque\Desktop\CODING KH\BUBT_LAB_ALGO
```

Worst Case – Reverse Sorted

```
C:\WINDOWS\system32\cmd.exe
5
5 4 3 2 1
Sorted: 1 2 3 4 5
Comparisons: 10
c:\Users\Kawsar Hoque\Desktop\CODING KH\BUBT_LAB_ALGO
```

Complexity Analysis

Best Case: $O(n^2)$ – Selection Sort always scans the full unsorted portion to find the minimum element, so the number of comparisons is always $n(n-1)/2$ regardless of input order.

Worst Case: $O(n^2)$ – The comparison count is identical to the best case; only the number of swaps differs between sorted and unsorted input.

3. Insertion Sort

● InsertionSort_270.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8      int arr[n];
9      for (int i = 0; i < n; i++)
10         cin >> arr[i];
11
12     int comparisons = 0;
13     for (int i = 1; i < n; i++)
14     {
15         int key = arr[i];
16         int j = i - 1;
17         while (j >= 0 && arr[j] > key)
18         {
19             comparisons++;
20             arr[j+1] = arr[j];
21             j--;
22         }
23         comparisons++;
24         arr[j+1] = key;
25     }
26     cout << "Sorted: ";
27     for (int i = 0; i < n; i++)
28         cout << arr[i] << " ";
29     cout << endl;
30     cout << "Comparisons: " << comparisons;
31     return 0;
32 }
```

Best Case – Already Sorted

```
C:\WINDOWS\system32\cmd.exe
5
1 2 3 4 5
Sorted: 1 2 3 4 5
Comparisons: 4
c:\Users\Kawsar Hoque\Desktop\CODING KH\BUBT_LAB_ALGO
```

Worst Case – Reverse Sorted

```
C:\WINDOWS\system32\cmd.exe
5
5 4 3 2 1
Sorted: 1 2 3 4 5
Comparisons: 14
c:\Users\Kawsar Hoque\Desktop\CODING KH\BUBT_LAB_ALGO
```

Complexity Analysis

Best Case: $O(n)$ – When the array is already sorted, the inner while loop never executes because each element is already in its correct position. Only $n-1$ comparisons are made.

Worst Case: $O(n^2)$ – When the array is reverse sorted, each element must be shifted past all previously sorted elements, resulting in $n(n-1)/2$ comparisons and shifts.

4. Binary Search

● BinarySearch_270.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8      int arr[n];
9      for (int i = 0; i < n; i++)
10         cin >> arr[i];
11
12     int key;
13     cin >> key;
14
15     int low = 0, high = n - 1;
16     int iterations = 0;
17     bool found = false;
18
19     while (low <= high)
20     {
21         iterations++;
22         int mid = (low + high) / 2;
23         if (arr[mid] == key)
24         {
25             cout << "Found at index " << mid;
26             found = true;
27             break;
28         }
29         else if (arr[mid] < key)
30             low = mid + 1;
31         else
32             high = mid - 1;
33     }
34     if (!found)
35         cout << "Not Found";
36     cout << endl << "Iterations: " << iterations;
37     return 0;
38 }
```


Best Case – Element Found at Middle

```
C:\WINDOWS\system32\cmd.exe
7
1 3 5 7 9 11 13
7
Found at index 3
Iterations: 1
c:\Users\Kawsar Hoque\Desktop\CODING KH\BUBT_LAB_ALGO
```

Worst Case – Element Not Found

```
C:\WINDOWS\system32\cmd.exe
7
1 3 5 7 9 11 13
6
Not Found
Iterations: 3
c:\Users\Kawsar Hoque\Desktop\CODING KH\BUBT_LAB_ALGO
```

Complexity Analysis

Best Case: $O(1)$ – The target element is found at the very first middle position, so only one iteration is needed.

Worst Case: $O(\log n)$ – When the element is not present or is located at an extreme end, the search space is halved with each iteration. For n elements, at most $\log_2(n)$ iterations are required.